Summarize

• Modeling Practice

- Conceptual modeling is used to capture ideas, understand realword system, specify solutions, and document key decisions to communicate efficietly. Softwtware shoul be organizeo into loosely coupled layers, such as User Interface, Communications and Operation system, where each layer uses abstractions provided by lower-lever layers.

Developers should prefer aggregation over inheritance because aggregation relationships can be altered dynamically at runtime, making them more flexible. It also emphasizes programming to an interface or abstraction to manage coupling and remove accidental complexity.

Testing Individual units of code should be evaluated with executable test cases to find desing errors, verify code correctness, document method usage, and enable regression testing

• UML

- UML is an expressive, semi-formal, and process-independent object oriented language that supports incremental development. System are modeled from three main perspective: the Analysis perspective (focus on real-word problem domains), the specification perspective, and the implementation perspective.

UML diagrams are divided into Structural diagrams (Class Object, Component, Deployment) that show static designs, and Behavioral diagrams (Use Case, Sequence Collaboration, Statechart, Activity) that emphasize dynamic views and event-ordered behaviors.

Diagramming Best Practices: Diagrams exist primarily to communicate, meaning developers should balance structural and behavioral models, avoid redundant diagrams, and organize elements spatially so conceptually linked items are physically close.

Basic Use Cases

- Actors are entities outside outside the system (users, hardware, or other systems) that interact with it, while Use Cases are discrete goals those actors need to accomplish, completly independent of how the software implements them. Use cases define the "what" and "why" whreas system interactions define the how.

Actors can be discovered by asking who will use the main funcitionally, who provides inputs, who uses outputs, or what external hardware the system controls.

Extend «extend» A operation where an extending use case alters or adds to a base goal under specific conditions via an extension point

Include «include» Used when one use case requires the completion of another use case to fulfill its goal.

· Class Diagrams

- Objects encapsulate state and behavior with distinct identities. Classes serve as abstractions for collections of objects sharing common characteristics. A class box is divided into compartments for its Name, Attributes, and Operations.

• Generalization: Represents is-a subset relationships, subtyping, or inheriting.

• Associations: Structural links between objects that can be uni-directional, bi-directional, or n-ary.

• Aggregation: A specific "is-part-of" association represented by a hollow diamond, where an object contains a subpart.

• Composition: A stricter form of aggregation where the subpart objects lifecycle is tied entirely to the existence of the aggregate object.

UML Quick Ref. G.

- The guide recommends modeling the real world problem first rather that jumping into software components, advising that the model's incompleteness should be used to trigger further domain questions. Provides explicit tables on drawing generalizations lines, naming actors, and using "include" to break down high-level goal versus "extend" for exceptional requirement conditions.

Describe how to map object modes of operation using guards, triggers and atomic actions, noting the usage of high-level states to describe complex activities and internal transitions for actions an objects takes without leaving its current state.